Problem 1 - EA for N-Queens - 40 points

Based on the description given in the lecture, your task is to implement Evolutionary Algorithm to solve the N-Queens Problem. Describe in details your crossover & mutation algorithms. Your implementation must be able to return the optimal solution for $N = 8$. Report your choice for hyperparameters including the size of population and the number of generations, and your fitness score in each generation.

N-queens problem -> N-queens problem makes us place 8 queens in such a way that there is no conflict neither in row, column nor in any diagonal.

In python code the Genetic Evolutionary Algorithms are used. Hyperparameters used:

```
self.Population = 10000          # 10000
self.Max_nGenerations = 100000000  # 100000000
self.nSwaps = 6
```

```
Size of the chessboard: 8
Generation 1
Fitness Score = 0
```

Implementation Details ->

1. Initialisation -> We have an overall population of 10,000 solutions. For solving the 8-Queens problem we have used EA.
2. Finding Fitness -> No of queens attacking each other. We have to minimise these attacks.
3. Selection -> In Selection process, 3 strategies are used in every generation.
    a. Fetched 2 best elements from Heap_Min. Heap_min saved the noOfAttacks for each Citizen and index. This will give us 2 elements with minimum attacks from the Heap.

    b. Random pairs -> Random citizens are chosen from the population. 2 queen position lists are chosen randomly.

    c. Best and worst pair -> best pair from min_heap and worst pair from max_heap are chosen.

4. **Crossover** -> There are 2 crossover algorithms used here.

    1. Half_Mother_Half_Father_Crossover -> In this type of crossover, we have 2 Parents the Mother and Father. From Father we take exactly N/2 queen positions and we take the remaining N/2 from queen positions of Mother.

       Eg. Mother = [3,1,6,2,5,7,0,4] Father = [2,5,1,7,4,0,6,3]
       N/2 = 4 -> Child = [3,1,6,2,4,0,6,3]
       There are two positions with column 3 so conflict has occured.

    2. Random Crossover -> There is a random integer called j which takes either 0 or 1. If random function produces j as 0 then we pick a solution from father else mother.

5. **Mutation Algorithms** -> In mutation we swap positions of queens. This helps us try new possibilities of positions.

Results

```
413     obj.main()
```

```
Size of the chessboard: 8


**Hyperparameters**
Size of Population =  100000
Max-Generations =  100000000
Number of swaps =  6
Size of chessboard =  8
Reached Init
Generation =  1
Minimum of attacking (Fitness Score):  0

Found successfully a solution:
[0, 2]
[1, 5]
[2, 7]
[3, 0]
[4, 3]
[5, 6]
[6, 4]
[7, 1]
```

Problem 2 - EA for TSP - 60 points

Based on the description given in the lecture, your task is to implement Evolutionary Algorithm to solve the Travelling Salesman Problem (TSP). Describe in details your crossover & mutation algorithms. Your implementation should be able to return the optimal solution for small N (let's say $N \leq 10$) that can be verified by a brute-force back-tracking algorithm or dynamic programming. Report your choice for hyperparameters including the size of population and the number of generations, and your fitness score in each generation.

For example, given these 4 points in 2-dimensional plane:
1 0 0
2 1 1
3 0 1
4 1 0

Q2.

CrossOver -> We have Father and Mother and both have city indexes. In Crossover we selected 2 random indices start position and end position and we take all the city indexes that are present in between these two positions from Parent 1. Remaining city indexes are taken from Parent 2.

Eg. Parent 1 = [3, 1, 4, 0, 2] Parent 13 = [2, 4, 1, 0, 3]
start = 1, end = 3
Child = [E, 1, 4, 0, E] [2, 1, 4, 0, 3]

Initially full array is marked as E. or None -> [E, E, E, E, E]
Traverse through Parent 1
1, 4, 0 are copied from Parent 1 at corresponding indices in child that is indices = 1, 2, 3 remaining 0 and 4 remain as None. -> [E, 1, 4, 0, E]

When we check Parent 2 we check each element. If element is present in child we don't insert. 2 is not present in Child so we inserted 2 at 0th position in child because 0th position is Empty.
4, 1, 0 is already present in child so we skip and 3 is not present in child so we insert 3 in child at the next Empty position.

Mutation -> We loop through 500 paths in our population one by one. We generate a random value between 0 and 1. We initialise a variable called mutation_rate as 0.6. And this value can be changed for better results. This is a hyperparameter. In our mutation logic we check if the randomly generated value is less than the mutation rate.

If the randomly generated value is less than the mutation rate, then we choose 2 random cities u and v and we swap those cities in one single route. We perform swap process for nSwap times.

If randomly generated value is greater then we keep it unchanged. We do the same process for remaining routes or paths in the list.

Results Obtained

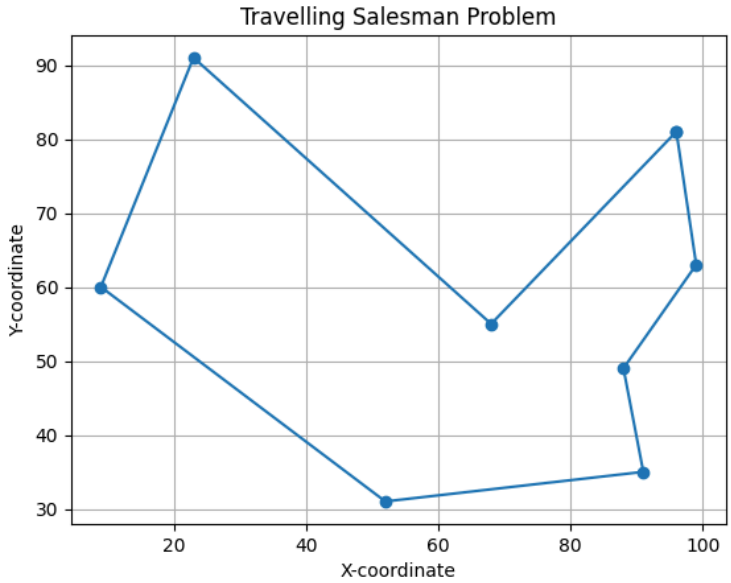
Small N

** Hyperparameters Used **
population size: 40
cities : 8
generations: 50
Select Percent: 30
Mutation Swaps: 2
Cities = [[52, 31], [23, 91], [96, 81], [68, 55], [99, 63], [88, 49], [91, 35], [9, 60]]

Best Distance Obtained is the fitness score

Generation 0: Best Distance Obtained = 270.97
Generation 1: Best Distance Obtained = 271.29
Generation 2: Best Distance Obtained = 269.26
Generation 3: Best Distance Obtained = 271.29
Generation 4: Best Distance Obtained = 271.29
Generation 5: Best Distance Obtained = 271.29
Generation 6: Best Distance Obtained = 271.29
Generation 7: Best Distance Obtained = 271.29
Generation 8: Best Distance Obtained = 271.29
Generation 9: Best Distance Obtained = 271.29
Generation 10: Best Distance Obtained = 271.29
Generation 11: Best Distance Obtained = 271.29
Generation 12: Best Distance Obtained = 271.29
Generation 13: Best Distance Obtained = 271.29
Generation 14: Best Distance Obtained = 271.29
Generation 15: Best Distance Obtained = 271.29
Generation 16: Best Distance Obtained = 271.29
Generation 17: Best Distance Obtained = 271.29
Generation 18: Best Distance Obtained = 271.29
Generation 19: Best Distance Obtained = 271.29
Generation 20: Best Distance Obtained = 271.29
Generation 21: Best Distance Obtained = 271.29
Generation 22: Best Distance Obtained = 271.29
Generation 23: Best Distance Obtained = 271.29
Generation 24: Best Distance Obtained = 271.29
Generation 25: Best Distance Obtained = 271.29

Generation 26: Best Distance Obtained = 271.29
Generation 27: Best Distance Obtained = 271.29
Generation 28: Best Distance Obtained = 271.29
Generation 29: Best Distance Obtained = 271.29
Generation 30: Best Distance Obtained = 271.29
Generation 31: Best Distance Obtained = 271.29
Generation 32: Best Distance Obtained = 271.29
Generation 33: Best Distance Obtained = 271.29
Generation 34: Best Distance Obtained = 271.29
Generation 35: Best Distance Obtained = 271.29
Generation 36: Best Distance Obtained = 271.29
Generation 37: Best Distance Obtained = 271.29
Generation 38: Best Distance Obtained = 271.29
Generation 39: Best Distance Obtained = 271.29
Generation 40: Best Distance Obtained = 271.29
Generation 41: Best Distance Obtained = 271.29
Generation 42: Best Distance Obtained = 271.29
Generation 43: Best Distance Obtained = 271.29
Generation 44: Best Distance Obtained = 271.29
Generation 45: Best Distance Obtained = 271.29
Generation 46: Best Distance Obtained = 271.29
Generation 47: Best Distance Obtained = 271.29
Generation 48: Best Distance Obtained = 271.29
Generation 49: Best Distance Obtained = 271.29



optimal distance = 271.2931711456921
optimal path = [2, 4, 5, 6, 0, 7, 1, 3]

Large N - XQF131

Attached in output files